

Grundlagen digitaler Systeme WS 25/26

Hausaufgabe 1

Abgabe: 06.11.2025, 23:59

Testate: 10.11.2025–21.11.2025

Bitte geben Sie in den folgenden Aufgaben ihre Lösungsschritte stets explizit an, falls nicht anders angegeben. Nicht kommentierte Lösungen werden **nicht** gewertet.

Aufgabe 1 *Signale, Abtasten und Quantisieren*

1. Entscheiden Sie für die folgenden Signale, ob sie wertkontinuierlich oder wertdiskret, sowie zeitkontinuierlich oder wertdiskret sind. Begründen Sie kurz.
 - Der Schall, den unsere Ohren hören.
 - Die Anzahl an Menschen in einem Museum.
 - Die Mittagstemperatur in Potsdam.
 - Die Anzahl verkaufter Produkte pro Tag.

Gegeben sei nun das folgende Signal (Abb. 1):

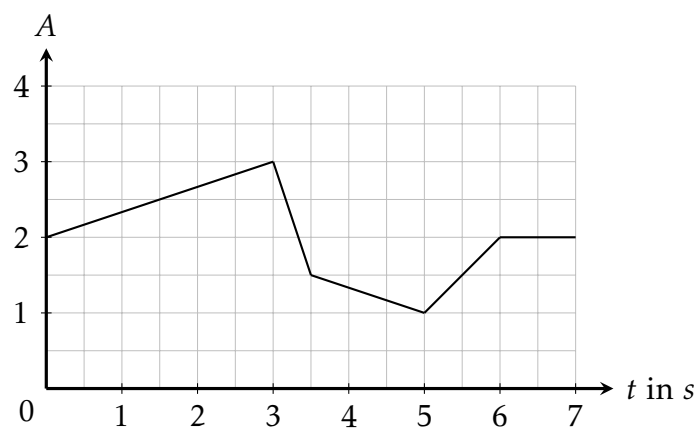


Abbildung 1: Verlauf des Signals A über der Zeit t

2. Berechnen Sie den absoluten Fehler (also den Unterschied des Flächeninhalts des Signals vor, und nach Abtasten und Quantisieren) für **zweifaches Abtasten pro Sekunde** und **Quantisierung auf 1/2 Einheiten**.
3. Betrachten Sie Ihr Ergebnis. Ist dieses zu erwarten gewesen? Ändert es sich, wenn das Signal nicht linear darstellbar ist? Begründen Sie anhand eines Beispiels.
4. Sie betrachten nun ein beliebigen Wert eines Signales, welcher sich zwischen zwei Quantisierungsstufen befindet. Was ist unter Annahme idealer Rundung der größtmögliche Quantisierungsfehler?

Aufgabe 2 Zahldarstellungen

Im folgenden gehen wir von einem 8-Bit Zweierkomplement-System aus.

1. Stellen Sie die Zahlen 36_{10} , -36_{10} und -128_{10} dar. Inwiefern ändert sich die Zahldarstellung, wenn man mehr als 8 Bits verwenden würde?
2. Bestimmen Sie die kleinste und größte darstellbare Zahl.
3. Erklären Sie, warum das Negieren der kleinsten Zahl nicht darstellbar ist.
4. Wir nennen das Negieren jedes Bits einer Zahl a folgend $NOT(a)$. Zeigen Sie, dass für alle Zahlen a im Zweierkomplement gilt: $-a = NOT(a) + 1$. *Hinweis: Überlegen Sie, was $NOT(a)$ mathematisch bedeutet.*

Folgend gehen wir von IEEE-754 single precision aus.

1. Stellen Sie die IEEE-754 Zahl C27FC000 als Dezimalzahl dar.
2. Stellen Sie die Fließkommazahl 42,875 als Binärzahl dar.

Aufgabe 3 UTF-8

1. Wie viele verschiedene Zeichen könnte man mit UTF-8 maximal darstellen? Recherchieren Sie, wie viele Zeichen wirklich mit UTF-8 darstellbar sind. Erklären Sie, warum in Praxis weniger Zeichen darstellbar sind als theoretisch möglich.
2. Gegeben sei folgender Ausschnitt eines Bytestreams: 41 C3 84 E4 B8 AD C3 A9 CE A9. Bestimmen Sie für jedes Byte, ob es ein Startbyte oder ein Folgebyte ist. Folgern Sie, wie viele Zeichen die Nachricht enthält.
3. UTF-8 ist ein selbstsynchronisierendes Kodierungsformat, das bedeutet es ist möglich, beim Lesen eines Bytestroms an einer beliebigen Stelle einzusteigen und die Grenzen der Zeichen zu erkennen. Erläutern Sie, wie dies möglich ist.
4. Erklären Sie, warum dasselbe nicht immer möglich ist, wenn man ohne Kenntnis der Bytegrenzen bei einem zufälligen Bit beginnt zu lesen.

Aufgabe 4 Erzeugendensysteme

1. Stellen Sie NAND nur durch NOR und NOR nur durch NAND dar. Was fällt auf?
2. Aus der Vorlesung wissen wir, dass NAND und NOR Erzeugendensysteme sind. Wir wollen nun noch andere Operator Mengen untersuchen. Entscheiden Sie für die folgenden Mengen, ob sie Erzeugendensysteme sind. Falls ja, stellen Sie NAND und NOR nur mittels der Grundschaltnetzen dieser Menge dar.
 - AND, OR
 - AND, NOT
 - XOR, OR
 - XOR, OR, TRUEHinweis: 'TRUE' ist ein Schaltnetz mit 0 Eingängen und einem Ausgang, der immer 1 sendet.
3. Gegeben sei die folgende logische Aussage:

$$((A \Leftrightarrow B) \Rightarrow \neg(C \wedge \neg A)) \wedge (A \vee \neg B)$$

Formen Sie diese in die DNF um. Vereinfachen Sie wenn möglich.

4. Geben Sie die Wahrheitstabelle zu Ihrer DNF an. Ist diese äquivalent zu der des ursprünglichen Ausdrucks?

Aufgabe 5 Prioritäts-Multiplexer

Wir haben in der Vorlesung gelernt, wie ein Multiplexer (MUX) funktioniert. Entwerfen Sie nun einen Prioritäts-Multiplexer. Dieser soll nicht wie der herkömmliche Multiplexer mit Dualzahl-Steuereingängen auswählen, welches Input am Output erscheint, sondern die folgende Strategie benutzen:

- Der PrioMUX hat genau so viele Steuereingänge wie Dateneingänge (genannt S_0 bis S_3).
 - Falls an jedem Steuereingang 0 anliegt, soll konstant 0 ausgegeben werden.
 - Falls an einem oder mehr Steuereingängen 1 anliegt, soll nur die Datenleitung mit dem Index der größten aktiven Steuerleitung ausgegeben werden. Diese hat also "Priorität".
1. Entwickeln sie das Schaltnetz für den PMux. Sie dürfen dabei alle bekannten Schaltnetze mit beliebig vielen (invertierten) Eingängen, sowie den herkömmlichen Multiplexer benutzen. Orientieren sie sich an der Vorlage.

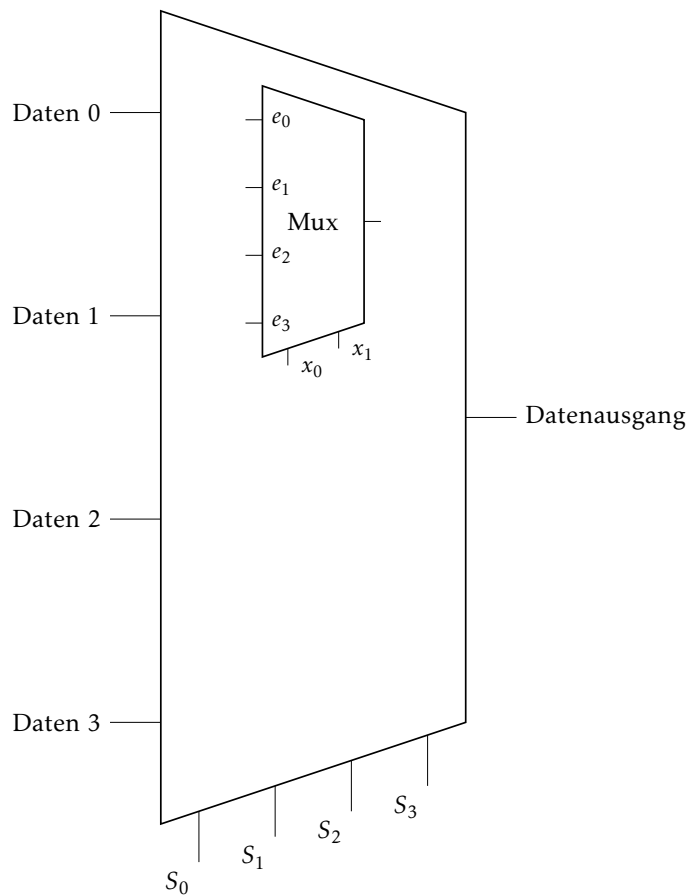


Abbildung 2: Vorlage

2. Wann wäre so ein Prioritäts-Multiplexer sinnvoll?

1.2

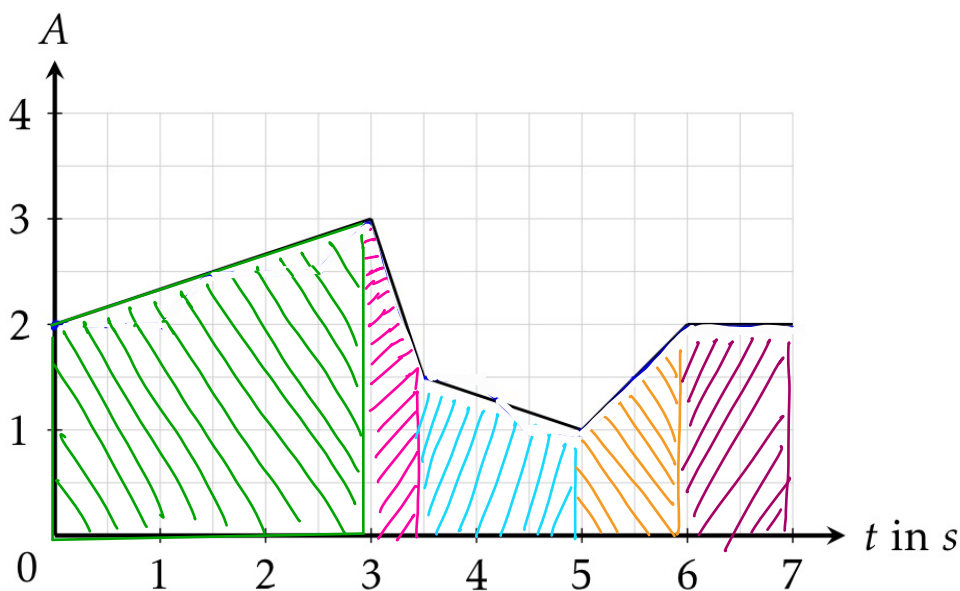


Abbildung 1: Verlauf des Signals A über der Zeit t

$$[0;3] = \frac{2,0 + 3,0}{2} \cdot 3 = 7,5$$

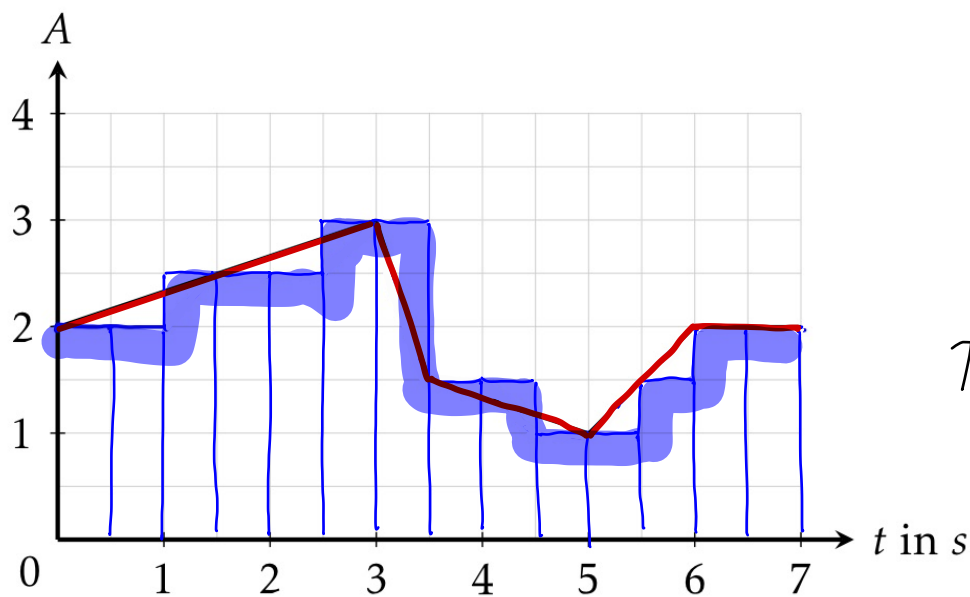
$$[3;3,5] = \frac{3,0 + 1,5}{2} \cdot 0,5 = 1,125$$

$$[3,5;5] = \frac{1,5 + 1,0}{2} \cdot 1,5 = 1,875$$

$$[5;6] = \frac{1,0 + 2,0}{2} \cdot 1 = 1,5$$

$$[6;7] = 2,0 \cdot 1,0 = 2$$

$$S_{\text{orig}} = 14,0$$



$$T = 0,5$$

Abbildung 1: Verlauf des Signals A über der Zeit t

$$[0; 0,5] = 2 \cdot 0,5 = 1$$

$$[0,5; 1] = 2 \cdot 0,5 = 1$$

$$[1; 1,5] = 2,5 \cdot 0,5 = 1,25$$

$$[1,5; 2] = 2,5 \cdot 0,5 = 1,25$$

$$[2; 2,5] = 2,5 \cdot 0,5 = 1,25$$

$$[2,5; 3] = 3 \cdot 0,5 = 1,5$$

$$[3; 3,5] = 3 \cdot 0,5 = 1,5$$

$$[3,5; 4] = 1,5 \cdot 0,5 = 0,75$$

$$[4; 4,5] = 1,5 \cdot 0,5 = 0,75$$

$$[4,5; 5] = 1 \cdot 0,5 = 0,5$$

$$[5; 5,5] = 1 \cdot 0,5 = 0,5$$

$$[5,5; 6] = 1,5 \cdot 0,5 = 0,75$$

$$[6; 6,5] = 2 \cdot 0,5 = 1$$

$$[6,5; 7] = 2 \cdot 0,5 = 1$$

$$S_{\text{quant.}} = 14,0$$

$$E_{\text{abs.}} = |S_{\text{quant.}} - S_{\text{orig.}}| = |14 - 14| = 0$$

Dieser Nullfehler bedeutet, dass die durch die Quantisierung und Abtastung verursachten Fehler das Volumen unter der Signalkurve idealerweise kompensiert haben. Obwohl an einzelnen Messpunkten Rundungsfehler auftraten, haben sich diese über den gesamten Integrationsbereich zu einer Abweichung von null ausgeglichen. Das Ergebnis zeigt, dass für dieses spezifische, linear angenäherte Signal die gewählte diskrete Repräsentation flächenmäßig exakt ist.

1.3

Aufgabe 1.3: Analyse des Ergebnisses

War das Ergebnis (Fehler = 0,0) zu erwarten? Nein, das ist nicht typisch, aber es ist in diesem idealen Modell möglich. Es passiert, wenn sich alle Rundungsfehler (Quantisierungsfehler) gegenseitig perfekt aufheben (kompensieren). Bei unserem Signal aus geraden Linien war dies zufällig der Fall.

Ändert es sich bei einem nicht-linearen Signal? Ja. Bei einem nicht-linearen Signal (z.B. einer Sinuskurve) ist ein perfekter Ausgleich der Fehler sehr unwahrscheinlich. Der Fehler wäre fast immer größer als Null. Wenn das Signal sich zu schnell ändert (schneller als die Abtastrate), wird der Fehler durch Aliasing sehr groß.

1.4

Aufgabe 1.4: Maximaler Quantisierungsfehler

Was ist der größtmögliche Quantisierungsfehler?

Die Quantisierungsstufe (der Abstand zwischen den Niveaus) beträgt $\Delta = 0,5$.

Der größte Fehler passiert, wenn der echte Signalwert genau in der Mitte zwischen zwei Stufen liegt (z.B. bei 1,25, das genau zwischen 1,0 und 1,5 liegt).

Bei idealer Rundung (Idealrundung) wird dieser Wert auf die nächste Stufe gerundet. Der maximale Fehler ist daher immer die Hälfte der Stufenbreite: $E(\text{max}) = \Delta / 2 = 0,5 / 2 = 0,25$.

(2.1) 8-Bit Zweierkomplement

• $36_{10} = 00100100$

• $-36_{10} = 11011100$

• $-128_{10} = 10000000$ (overflow)

Wenn wir mehr als 8 Bit haben, fügen wir Nullen recht hinzu.

(2.2) Die größte Zahl:

$01111111 = 127$

$0 \cdot 128 + 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 127$

Die kleinste Zahl:

$10000000 = -128$

(2.3) Wenn wir -128 negieren, passiert overflow. 8 Bit ist nicht genug für die Zahl 128.

(2.4) $-a = \text{NOT}(a) + 1$

Wenn $a + (\text{NOT}(a) + 1) = 0$, dann $\text{NOT}(a) + 1 = -a$

Sei $a = 36 = 00100100$,

$\text{NOT}(a)$ — invertierende $a = 11011011$

$a + \text{NOT}(a) = 11111111 = -1$

$(a + \text{NOT}(a)) + 1 = -1 + 1$

$a + (\text{NOT}(a) + 1) = 0$

(2.5) 1) C27FC000

C — 1100

2 — 0010

7 — 0111

F — 1111

C — 1100

0 — 0000

0 — 0000

0 — 0000

S — 1 = 1

E — 8 = 10000 op 5 (Potenz)

M — 23 = 1.1111111100000000000000

„0“ kann man weglassen am Ende

$10000100 = 128 + 4 = 132$

Bias = 127 (für 32 Bit) $\Rightarrow 132 - 127 = 5$

(M) hidden bit (спрятанный)

Erste 1

36
 $36 : 2 = 18$
 $18 : 2 = 9$
 $9 : 2 = 4$
 $4 : 2 = 2$
 $2 : 2 = 1$
 $1 : 2 = 0$

0
 0
 1
 0
 0
 1

-36

1) $36 = 00100100$
 2) invertieren: 11011011
 3) +1 (1+1=0, 1 8 yme!)

11011011
 $+ 00100001$
 11011100

-128

$128 : 2 = 64$ 0 b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0
 $64 : 2 = 32$ 0 -128 64 32 16 8 4 2 1
 $32 : 2 = 16$ 0 1 0 0 0 0 0 0
 $16 : 2 = 8$ 0
 $8 : 2 = 4$ 0
 $4 : 2 = 2$ 0
 $2 : 2 = 1$ 0
 $1 : 2 = 0$ 1

Addieren, um 0 zu bekommen

$$1.11111111 \cdot 2^5 = \cdot 10^5 \text{ (двигаем запятую на 5 знаков вправо)}$$

↑
умножить двоичн число на 2^5 — это то же самое, что в десятичн. системе $\cdot 10^5$

$$\underbrace{111111}_1 \cdot \underbrace{1111}_2 = 63,9375$$

$$1) \quad 1 \ 1 \ 1 \ 1 \ 1 \ 1 \\ 32+16+8+4+2+1 = 63$$

2) Bruchteil-Umwandlung

$$\dots \cdot 1111 \\ \frac{1}{2} \ \frac{1}{4} \ \frac{1}{8} \ \frac{1}{16} \\ 0,5 \ 0,25 \ 0,125 \ 0,0625 = 0,9375$$

$$2) \quad 42,875$$

S — " + "

$$\begin{array}{l} 42 = 101010 \\ 42 : 2 = 21 \quad 0 \\ 21 : 2 = 10 \quad 1 \\ 10 : 2 = 5 \quad 0 \\ 5 : 2 = 2 \quad 1 \\ 2 : 2 = 1 \quad 0 \\ 1 : 2 = 0 \quad 1 \end{array} \quad \uparrow$$

0,875 (сверху вниз)

$$\begin{array}{l} 0,875 \cdot 2 = 1,75 \quad 1 \text{ (1 пишем, } 0,75 \text{ переносим)} \\ 0,75 \cdot 2 = 1,5 \quad 1 \text{ (1 пишем, } 0,5 \text{ переносим дальше)} \\ 0,5 \cdot 2 = 1 \quad 1 \text{ (0 остался. концы)} \end{array}$$

$$\underbrace{101010}_1 \cdot 111 = 1.01010111$$

Normalisieren: Заметая г. ставим после первого числа. Значит надо сдвинуть на 5 позиций влево. Это значит, что реальный показ. степени $e = 5$.

$$\bullet \quad e + \text{Bias} = 5 + 127 = 132$$

$$132 - 6 \text{ 8 Bit} = 10000100$$

$$\begin{array}{l} 132 : 2 = 66 \quad 0 \\ 66 : 2 = 33 \quad 0 \\ 33 : 2 = 16 \quad 1 \\ 16 : 2 = 8 \quad 0 \\ 8 : 2 = 4 \quad 0 \\ 4 : 2 = 2 \quad 0 \\ 2 : 2 = 1 \quad 0 \\ 1 : 2 = 0 \quad 1 \end{array}$$

• $M (23 \text{ Bit}) = \cancel{1} 01010111$ (по 1 единица — скрытая, убираем)

01010111 — добиваем до 23 Bit, ставим „0“ в конце.

S

E

M

0 10000100 010101110000000000000000